University of Toronto
CSC343, Winter 2025

Assignment 1: Pranav Rao (1008845065) & Eleonora Scognamiglio (1008779920)

Delete this before you hand in

Unary operators on relations:

- $\Pi_{x,y,z}(R)$
- $\sigma_{condition}(R)$
- $\rho_{New}(R)$

Binary operators on relations:

- $R \times S$
- $R \bowtie S$
- $R \bowtie_{condition} S$
- $R \cup S$
- $R \cap S$
- $R - S$

Logical operators:

- $\vee$
- $\wedge$
- $\neg$

Assignment:

- $New(a, b, c) := R$

General LaTeX tips:

- Some sequences of letters look terrible in "math mode" (inside dollar signs). For example, `$Offer$` comes out as $Offer$. You can use `\var` to fix that. For example, `$\var{Offer}$` comes out as Offer.
- When an expression is deeply nested, it can be hard to match the brackets with your eyes. You can vary the size of the brackets to make that easier. This LaTeX:

  ```
  \Bigg(  \bigg[   \Big(  \big(   \big)  \Big)  \bigg]  \Bigg)
  ```

  produces this result:

  $$\Bigg(\bigg[\Big(\big(\big)\Big)\bigg]\Bigg)$$

- Some of your expressions will become very wide because there are multiple, long conditions on a select. You can stack those conditions using `\substack`. Here's an example:

$$\sigma_{\substack{x<20 \\ \wedge \\ y=10}}(\textit{Some big expression})$$

  Notice that this requires using the package called "amsmath", which is accomplished by saying `\usepackage{amsmath}`.

Below we've included names for the assignment questions, and a nonsense example of how a query might look in LaTeX. As required in this assignment, it uses "–" to indicate comments. Note that we have added less vertical space between comments and the algebra they pertain to than between steps in the algebra. This helps the comments visually stick to the algebra.

Part 1: Create an instance

City

id	name	country
1	Mississauga	Canada
2	Rome	Italy
3	Frankfurt	Germay

Airline

code	name
AC	Air Canada
LH	Lufthansa

Passenger

id	firstName	surName
1	Sadia	Sharmin
2	Sadia	Sharmin
3	Sadia	Sharmin

Plane

tailNumber	model	airline
1	Airbus A380-800	AC
2	Boeing 777	LH
3	Boeing 787 Dreamliner	LH

Passenger

id	plane	row	letter	class
1	1	22	F	Economy
2	2	8	A	First
3	3	25	C	Economy

Airport

code	name	city
YYZ	Toronto Pearson International Airport	1
FCO	Leonardo da Vinci International Airport	2
FRA	Frankfurt Airport	3

Route

flightNumber	airline	source	destination
AC890	AC	YYZ	FCO
LH231	LH	FCO	FRA
LH470	LH	FRA	YYZ

Flight

id	route	plane	schedDeparture	schedArrival
1	AC890	1	5/1/2025 23:45:00	5/2/2025 8:15:00
2	LH231	2	5/22/2025 8:00:00	5/22/2025 10:00:00
3	LH470	3	5/22/2025 11:55:00	5/22/2025 20:20:00

Departure

flight	dateTime
1	5/1/2025 23:50:00

Arrival

flight	dateTime
1	5/2/2025 8:25:00

FlightPrice

flight	class	Price

Booking

id	passenger	seat	flight	price	dateTime
1	1	1	1	600	12/6/2024 21:00:00
2	2	2	2	950	5/9/2025 9:00:00
3	3	3	3	1200	5/9/2025 9:00:00

Part 2: Give the output of a query

The output of the query on our instance of the database is as follows:

id	passenger	seat	flight
1	1	1	1
3	3	3	3

Part 3: Violating a constraint

City

id	name	country
1	Mississauga	Canada
2	Rome	Italy

Airline

code	name
AC	Air Canada

Passenger

id	firstName	surName
1	Sadia	Sharmin

Plane

tailNumber	model	airline
1	Airbus A380-800	AC

Passenger

id	plane	row	letter	class
1	1	22	F	Economy
2	2	8	A	First

Airport

code	name	city
YYZ	Toronto Pearson International Airport	1
FCO	Leonardo da Vinci International Airport	2

Route

flightNumber	airline	source	destination
AC890	AC	YYZ	FCO

Flight

id	route	plane	schedDeparture	schedArrival
1	AC890	1	5/1/2025 23:45:00	5/2/2025 8:15:00

Departure

flight	dateTime
1	5/1/2025 23:50:00

Arrival

flight	dateTime
1	5/2/2025 8:25:00

FlightPrice

flight	class	Price

Booking

id	passenger	seat	flight	price	dateTime
1	1	1	1	600	12/6/2024 21:00:00
2	1	2	1	950	5/9/2025 9:00:00

Part 4: Queries

Q1

Cannot be expressed.

Q2

-- The flight ID and route of all delayed flights in 2024.

$$DelayedFlights(id, route) := \pi_{\substack{f.departure,\\ f.route}} \left(\sigma_{\substack{f.schedDeparture.year=2024 \wedge \\ d.dateTime \geq f.schedDeparture+=1\text{hour} \wedge \\ f.id=d.flight}} (\rho_f Flight \times \rho_d Departure) \right)$$

-- IDs of delayed flights with the code of their airline.

$$AirlineDelay(id, airline) := \pi_{df.id, route.airline} (\sigma_{r.flightNumber=df.route} (\rho_{df} DelayedFlights \times \rho_r Route))$$

-- The ID and airline of one flight for every airline that has three or more delays.

$$ThreeDelays(id, airline) := \pi_{\substack{A1.id,\\ A1.airline}} \left(\sigma_{\substack{A1.airline \\ =A2.airline \\ =A3.airline \wedge \\ A1.id \neq A2.id \neq A3.id}} (\rho_{A1} AirlineDelay \times \rho_{A2} AirlineDelay \times \rho_{A3} AirlineDelay) \right)$$

-- All flights from airlines that are delayed

$$FlightsFromDelayed(id, airline, schedDeparture) := \pi_{td.id, td.airline, f.schedDeparture} (\rho_{td} ThreeDelays \times \rho_f Flight)$$

-- The ID, airline code, and scheduled departure of all flights from airlines that are delayed that are NOT the latest scheduled flights

$$NotMax(id, code, schedDeparture) :=$$

$$\pi_{\substack{D1.id,\\ D1.code,\\ D1.schedDeparture}} \left(\sigma_{\substack{D1.schedDeparture \\ <D2.schedDeparture \\ \wedge D1.airline \\ =D2.airline}} (\rho_{D1} FlightsFromDelayed \times \rho_{D2} FlightsFromDelayed) \right)$$

-- The airline code, the airline name, and the ID of the flight number that is the most recently delayed flight for all airlines that have at least three delays in 2024.

$$\pi_{code, name, id} ((FlightsFromDelayed - NotMax) \bowtie Airline)$$

Q3

-- IDs of all cities that have an airport

$$HasAirport(cityID) := \pi_{city} (Airport)$$

-- IDs of all cities that have a route ending in that city

$$HasRouteEnding(cityID) := \pi_{a.city}\left(\sigma_{r.destination,a.code}\left(\rho_r Route \times \rho_a Airport\right)\right)$$

-- IDs of all cities that do NOT have a route ending in that city

$$HasNoRouteEnding(id) := HasAirport - HasRouteEnding$$

-- Name and city of all cities that do not have a route ending at that city

$$\pi_{c.name,c.city}\left(\rho_c City \bowtie HasNoRouteEnding\right)$$

Q4

-- Flight ID, seat class, and seat price of every seat in every flight where the seat is NOT the cheapest seat in its flight.

$$NotCheap(flight, class, price) := \pi_{\substack{T1.flight,\\ T1.class,\\ T1.price}}\left(\sigma_{\substack{T1.flight=T2.flight\wedge\\ T1.price>T2.price}}\left(\rho_{T1} FlightPrice \times \rho_{T2} FlightPrice\right)\right)$$

-- Flight ID and price of the cheapest seat in each flight

$$Cheap(flight, price) := \pi_{flight,price}\left(FlightPrice - NotCheap\right)$$

-- Flight number, source, destination, flight id, and airline of every Flight:

$$RouteAndFlight(flightNum, source, destination, flightId, airline) :=$$
$$\pi_{\substack{r.flightNumber,\\ r.source,\\ r.destination,\\ f.id,\\ f.airline}}\left(\sigma_{r.flightNumber=flight.route}\left(\rho_r Route \times \rho_f Flight\right)\right)$$

-- Flight number, source, destination, flight id, airline, and the price of the cheapest seat on every Flight:

$$AllFlightInfo(flightNum, source, destination, flightId, airline, price) :=$$
$$\pi_{\substack{r.flightNumber,\\ f.source,\\ f.destination,\\ f.flightId,\\ f.airline\\ c.price}}\left(\sigma_{f.flightId=c.flight}\left(\rho_f RouteAndFlight \times \rho_c Cheap\right)\right)$$

-- Flight IDs and numbers of pairs of flights that have the same source and destination, with the cheaper one reported first OR the one with the lower flight ID reported first if the prices are tied

$$\pi_{\substack{R1.flightId,\\ R1.flightNumber,\\ R2.flightId,\\ R2.flightNumber}}\left(\sigma_{\substack{R1.source=R2.source\wedge\\ R1.destination=R2.destination\wedge\\ R1.airline\neq R2.airline\wedge\\ [R1.price<R2.price\vee\\ R1.price=R2.price\wedge\\ R1.flightId<R2.flightId)]}}(\rho_{R1}AllFlightInfo \times \rho_{R2}AllFlightInfo)\right)$$

Q5

-- Scheduled departure and arrival of direct flights from YYZ to LIS arriving after June 17, 2025 at 11:00 AM

$$DirectFlights(schedD, schedA) := \pi_{\substack{r.schedDeparture,\\ r.schedArrival}}\left(\sigma_{\substack{f.route=r.flightNumber\\ \wedge r.source=\text{YYZ}\\ \wedge r.destination=\text{LIS}\\ \wedge f.schedArrival\\ >\text{June 17, 2025, 11:00}}}(\rho_r Route \times \rho_f Flight)\right)$$

-- The flight ID, scheduled departure, arrival, source, and destination of all flights starting from YYZ

$$FromYYZ(id, schedD, schedA, src, dest) := \pi_{\substack{f.id,\\ f.schedDeparture,\\ f.schedArrival,\\ r.source,\\ r.destination}}\left(\sigma_{\substack{f.route=r.flightNumber\\ \wedge r.source=\text{YYZ}}}(\rho_r Route \times \rho_f Flight)\right)$$

-- The flight ID, scheduled departure, arrival, source, and destination of all flights going to LIS arriving after June 17, 2025 at 11:00

$$ToLIS(id, schedD, schedA, src, dest) := \pi_{\substack{f.id,\\ f.schedDeparture,\\ f.schedArrival,\\ r.source,\\ r.destination}}\left(\sigma_{\substack{f.route=r.flightNumber\\ \wedge r.destination=\text{LIS}\\ \wedge f.schedArrival>\text{June 17, 2025, 11:00}}}(\rho_r Route \times \rho_f Flight)\right)$$

-- Scheduled departure and arrival of flights going from YYZ to LIS, stopping over at an intermediate airport for between 1 to 24 hours inclusive and arriving at LIS after June 17, 2025 at 11:00

$$Layovers(schedD, schedA) := \pi_{y.schedD, l.schedA}\left(\sigma_{y.dest=l.src\wedge 1\leq l.schedD-y.schedA\leq 24}(\rho_y FromYYZ \times \rho_l ToLIS)\right)$$

-- Direct flights and flights with exactly one layover going from YYZ to LIS arriving in LIS after June 17, 2025 at 11:00 and if applicable having a layover of between 1 and 24 hours, inclusive

$$DirectFlights \cup Layovers$$

Q6

-- IDs of all possible pairs of passengers who have booked a flight

$$AllPairs(p1, p2) := \sigma_{T1.pass < T2.pass} \left(\rho_{T1}\pi_{passenger}\left(Booking\right) \times \rho_{T2}\pi_{passenger}\left(Booking\right)\right)$$

-- IDs for every pair of passengers that have travelled at least one flight separately (avoiding pseudoduplicates)

$$TraveledSeparate(p1, p2) := \pi_{\substack{T1.passenger,\\ T2.passenger}} \left(\sigma_{\substack{T1.flight \neq T2.flight\\ \wedge T1.passenger < T2.passenger}} \left(\rho_{T1}Booking \times \rho_{T2}Booking\right)\right)$$

-- IDs for pairs of passengers who have travelled every flight they've taken together

$$TraveledTogether(p1, p2) := AllPairs - TraveledSeparate$$

-- Booking and flight info for every booking

$$BookingAndFlight(bookingId, passenger, seat, flight, price, dateTime, flightId, plane, schedDeparture, schedArrival) := \sigma_{b.flight = f.id}\left(\rho_b Booking \times \rho_f Flight\right)$$

-- Every passenger (by ID) mapped to all flights that are NOT the latest departing flight for that passenger

$$NotMax(bookingId, passenger, seat, flight, price, dateTime, flightId, plane, schedDeparture, schedArrival) := \pi_{\substack{T1.bookingId,\\ T1.passenger,\\ T1.seat,\\ T1.flight,\\ T1.price,\\ T1.dateTime,\\ T1.flightId,\\ T1.plane,\\ T1.schedDeparture,\\ T1.schedArrival}} \left(\sigma_{\substack{T1.schedDeparture < T2.schedDeparture\\ \wedge T1.passenger = T2.passenger}} \left(\rho_{T1}BookingAndFlight \times \rho_{T2}BookingAndFlight\right)\right)$$

-- Passenger ID and the ID of the latest flight they took

$$LatestFlight(p1, flight) := \pi_{passenger, flight}\left(BookingAndFlight - NotMax\right)$$

-- Passengers that have the same last name and always travel together, reported by their two first names, their common last name, and the ID of their most recent flight

$$\pi_{\substack{T2.firstName,\\ T3.firstName,\\ T2.surName,\\ T1.flight}} \left(\sigma_{\substack{T1.p1 = T2.id\\ \wedge T1.p2 = T3.id\\ \wedge T2.surName = T3.surName}} \left(\rho_{T1}(LatestFlight \bowtie TraveledTogether) \times \rho_{T2}Passenger \times \rho_{T3}Passenger\right)\right)$$

Q7

-- Tail number of planes that were used for at least four different flights

$$UsedFourTimes(tailNumber) = \pi_{T1.plane}\left(\sigma_{\substack{T1.id \\ \neq T2.id \\ \neq T3.id \\ \neq T4.id \\ \wedge T1.plane \\ = T2.plane \\ = T3.plane \\ = T4.plane}}(\rho_{T1}Flight \times \rho_{T2}Flight \times \rho_{T3}Flight \times \rho_{T4}Flight)\right)$$

-- Tail number of planes that were used for less than four different flights

$$NotUsedFourTimes(tailNumber) := \pi_{tailNumber}(Plane - UsedFourTimes)$$

-- Tail number and airline of planes that were used for less than four different flights

$$\pi_{tailNumber,airline}(NotUsedFourTimes \bowtie Plane)$$

Q8

-- ID, passenger, flight ID, seat ID, and seat row for every booking

$$BookingSeat(bookingId, passenger, flight, seat, seatRow) := \pi_{\substack{b.id, \\ b.passenger, \\ b.flight, \\ b.seat, \\ s.row}}(\sigma_{b.seat=s.id}(\rho_b Booking \times \rho_s Seat))$$

-- IDs of passengers who have booked at least three seats on the same flight

$$AtLeastThree(id) := \pi_{B1.passenger}\left(\sigma_{\substack{B1.passenger \\ = B2.passenger \\ = B3.passenger \\ \wedge B1.flight \\ = B2.flight \\ = B3.flight \\ \wedge B1.seat \\ \neq B2.seat \\ \neq B3.seat}}(\rho_{B1}BookingSeat \times \rho_{B2}BookingSeat \times \rho_{B3}BookingSeat)\right)$$

-- IDs of passengers who have booked at least two seats on the same flight but the seats are not in the same row

$$AtLeastTwo(id) := \pi_{B1.passenger}\left(\sigma_{\substack{B1.passenger \\ =B2.passenger \\ \wedge B1.flight \\ =B2.flight \\ \wedge B1.seat \\ \neq B2.seat \\ \wedge B1.row \neq B2.row}}(\rho_{B1}BookingSeat \times \rho_{B2}BookingSeat)\right)$$

-- Passengers who have booked at least one flight and for all of their booked flights have booked exactly two seats, but the two seats are not in the same row

$$AtLeastThree - AtLeastTwo$$

Q9

Cannot be expressed.

Q10

-- Passenger, seat, flight ID, price, and seat class for every booking

$$BookingAndSeat(passenger, seat, flight, bookingPrice, class) :=$$
$$\pi_{b.passenger, b.seat, b.flight, b.price, s.class}(\sigma_{s.id=b.seat}(\rho_b Booking \times \rho_s Seat))$$

-- Passenger, seat, flight ID, price, seat class, and the actual price for that seat for every booking

$$AllInfo(passenger, seat, flight, bookingPrice, class, price) := BookingAndSeat \bowtie FlightPrice$$

-- IDs of all passengers that paid an amount greater or equal to than the actual seat price during their booking

$$PaidMore(passenger) := \pi_{passenger}(\sigma_{bookingPrice \geq price}(AllInfo))$$

-- Everyone who paid strictly less than the price recorded in FlightPrice for every flight they have taken

$$\pi_{passenger}(Booking) - PaidMore$$

Q11

-- Seat, flight ID, price, and seat class for every booking

$$BookingAndSeat(seat, flight, bookingPrice, class) :=$$
$$\pi_{b.seat, b.flight, b.price, s.class}(\sigma_{s.id=b.seat}(\rho_b Booking \times \rho_s Seat))$$

-- Seat, flight ID, price, seat class, and the actual price for that seat for every booking

$$AllInfo(seat, flight, bookingPrice, class, price) := BookingAndSeat \bowtie FlightPrice$$

-- IDs of all the flights that would make a route "unpopular" (2 seats booked with a 50% or higher discount)

$$UnpopularFlights(id) := \pi_{T1.flight}\left(\sigma_{\substack{T1.flight=T2.flight \\ \wedge T1.seat\neq T2.seat \\ \wedge T1.bookingPrice\leq 0.5\cdot T1.price \\ \wedge T2.bookingPrice\leq 0.5\cdot T2.price}}(\rho_{T1}AllInfo \times \rho_{T2}AllInfo)\right)$$

-- The routes (sources and destinations) containing the unpopular flights i.e. unpopular routes

$$UnpopularRoutes(source, dest) := \pi_{\substack{r.source, \\ r.destination}}\left(\sigma_{\substack{T1.route \\ =r.flightNumber}}(\rho_{T1}(UnpopularFlight \bowtie Flight) \times \rho_r Route)\right)$$

-- Airline code, source, and destination of every possible combination of airline and unpopular routes

$$AirlineUnpopular(airline, source, dest) := \pi_{a.code}(\rho_a Airline \times UnpopularRoutes)$$

-- Combinations of airline code, source, and destination where the airline actually flew the route

$$AirlineRoutesFlown(airline, source, dest) := \pi_{airline,source,destination}(Route)$$

-- Combinations of airline code, source, and destination where the airline actually flew the route AND the route was unpopular

$$UnpopularRoutesFlown(airline, source, dest) := AirlineUnpopular \cap AirlineRoutesFlown$$

-- Combinations of airline code, source, and destinatoin where the route as unpopular but the airline did NOT fly it

$$UnpopularRoutesNotFlown(code, source, dest) := AirlineUnpopular - UnpopularRoutesFlown$$

-- Airline codes of airlines that have no unpopular routes NOT flown (i.e. airline codes of airlines that have operated a flight on all unpopular route)

$$\pi_{code}(Airline) - \pi_{code}(UnpopularRoutesNotFlown)$$

Part 5: Additional Integrity Constraints

Q1

-- All consecutive pairs of rows

$$ConsecutivePairs(T_1.id, T_1.row, T_2.id, T_2.row) := \pi_{T_1.id,T_1.row,T_2.id,T_2.row}(\sigma_{T_1.row=T_2.row-1}(\rho_{T_1}Seat \times \rho_{T_2}Seat))$$

-- All the rows, where the columns are unioned together to ensure that the first and last rows are included.

$$ConsecutiveRows(row) := (\pi_{T_1.row}(ConsecutivePairs)) \cup (\pi_{T_2.row}(ConsecutivePairs))$$

-- Ensure that all seats coincide with the consecutive rows and that row number one is included

$$((\pi_{row}(Seats)) - ConsecutivePairs) \cup (\{(1)\} - \sigma_{row=1}(ConsecutiveRows)) = \emptyset$$

Q2

-- Merge all tables necessary to combine all required information. This can be used directly in the next steps

$$AllInfo(BookingTime, schedDeparture, actualDeparture, actualArrival, id) := \pi_{Booking.dateTime, Flight.schedDeparture, Departure.date}$$

$$AllInfo(bookingTime, schedDeparture, actualDeparture, actualArrival, id)) :=$$

$$\pi_{\substack{Booking.dateTime,\\ Flight.schedDeparture,\\ Departure.dateTime,\\ Arrival.dateTime, Flight.id}} \left(\sigma_{\substack{Flight.id=Booking.flight\\ =Departure.flight\\ =Arrival.flight}} (Flight \times Booking \times Departure \times Arrival) \right)$$

-- Now we directly select by all the conditions listed in the description of the constraint

$$\sigma_{\substack{bookingTime \geq schedDeparture \vee\\ actualDeparture < schedDeparture \vee\\ actualArrival \leq actualDeparture}} (AllInfo) = \emptyset$$

Q3

-- Combine the information about flights and planes

$$FlightAndPlane(flightID, route, plane, schedDeparture, schedArrival, model, airline) :=$$

$$\pi_{\substack{Flight.id, Flight.route, Flight.plane,\\ Flight.schedDeparture, Flight.schedArrival,\\ Plane.model, Plane.airline}} (\sigma_{Flight.plane=Plane.tailNumber}(Flight \times Plane))$$

-- Here we are creating a mapping, where each flight (in T_1) is mapped to all flights (in T_2) that happened before it using the same plane.

$$AllMappings(plane, T_1.flightID, T_1.schedD, T_1.schedA, T_1.route, T_2.flightID, T_2.schedD, T_2.schedA, T_2.route) :=$$

$$\pi_{\substack{T_1.plane, T_1.flightID, T_1.schedD,\\ T_1.schedA, T_1.route, T_2.flightID,\\ T_2.schedD, T_2.schedA, T_2.route}} \left(\sigma_{\substack{T_1.plane=T_2.plane \wedge\\ T_1.schedD > T_2.schedA}} (\rho_{T_1} FlightAndPlane \times \rho_{T_2} FlightAndPlane) \right)$$

-- Here, for each flight, we filter for all the flights on the same plane that happened before it, except for the one that happened latest

$$NotMax(plane, T_1.flightID, T_1.schedD, T_1.schedA, T_1.route, T_2.flightID, T_2.schedD, T_2.schedA, T_2.route) :=$$

$$\pi_{\substack{A_1.T_1.plane, A_1.T_1.flightID, A_1.T_1.schedD,\\ A_1.T_1.schedA, A_1.T_1.route, A_1.T_2.flightID,\\ A_1.T_2.schedD, A_1.T_2.schedA, A_1.T_2.route}} \left(\sigma_{\substack{A_1.T_2.schedA < A_2.T_2.schedA \wedge\\ A_1.plane=A_2.plane}} (\rho_{A_1} AllMappings \times \rho_{A_2} AllMappigs) \right)$$

-- By subtracting all mapped flights except for the latest one from all mapped flights, we find a "mapping" from each flight to the latest previous flight that used the same plane.

$$LastArrivedMap(place, flightID1, flightID2, schedD1, schedA1, schedD2, schedA2, route1, route2) := AllMappings - NotMax$$

-- Now we select for the two constraints (airport constraint and layover time constraint) and we ensure that the set is empty.

$$\sigma_{\substack{LastArrivedMap.route1=R_1.flightNumber \wedge \\ LastArrivedMap.route2=R_2.flightNumber \wedge \\ R_1.dest \neq R_2.source \wedge \\ LastArrivedMap.schedA1+2hours<LastArrivedMap.schedD2}} (LastArrivedMap \times \rho_{R_1} Route \times \rho_{R_2} Route) = \emptyset$$

Q4

Cannot be expressed

Q5

-- Gather information on classes by combining flights and seats information.

$$ClassesOnFlights(flight, class) := \pi_{Flight.id, Seat.class} (\sigma_{Seat.plane=Flight.plane} (Seat \times Flight))$$

-- Compare all the information we have about classes with the information recorded in $FlightPrice$. Make sure that the two sets are equal

$$[FlightPrice - (ClassesOnFlight \bowtie FlightPrice)] \cup [(ClassesOnFlight \bowtie FlightPrice) - FlightPrice] = \emptyset$$

Q6

-- Combine information about the airports and cities

$$AriportAndCity(code, country) := \pi_{Airport.code, City.country} (\sigma_{Airport.city=City.id} (Airport \times City))$$

-- Find all domestic routes, i.e., all routes whose source and destination are in the same country

$$DomesticRoutes(Route) :=$$

$$\pi_{Route.flightNumber} \left(\sigma_{\substack{Route.source=AC_1.code \wedge \\ Route.destination=AC_2.code \wedge \\ AC_1.country=AC_2.country}} (Route \times \rho_{AC_1} AirportAndCity \times \rho_{AC_2} AirportAndCity) \right)$$

-- Combine this with flight information to obtain all domestic flights, i.e. flights with a route whose source and destination are in the same country.

$$DomesticFligths(plane, schedD) := \pi_{plane, schedDeparture} (DomesticRoutes \bowtie Flights)$$

-- Obtain all international flights, by removing domestic flights from all flights

$$InternationalFlights(plane, schedD) := (\pi_{plane, schedD} (Flight)) - DomesticFlights$$

-- Find planes that have been used for both domestic and international flights

$$PlanesInBoth(plane) := (\pi_{plane} (DomesticFlights)) \cap (\pi_{plane} (InternationalFlights))$$

-- Find all flights that use planes only used in domestic flights

$$OnlyDomesticPlanes(plane, schedD) := ((\pi_{planes}(DomesticFlights)) - PlanesInBoth) \bowtie DomesticFlights$$

-- Find planes that are exclusively used for domestic flights and that have flown at least four times in a month. Here for conciseness, we abbreviate the name $OnlyDomesticPlanes$ as ODP and $schedD$ as SD.

$$AtLeastFour(plane, schedD) :=$$

$$\pi_{T_1.plane, T_1.SD}\left(\sigma_{\substack{T_1.SD.year=T_2.SD.year=T_3.SD.year=T_4.SD.year\wedge \\ T_1.SD.month=T_2.SD.month=T_3.SD.month=T_4.SD.month\wedge \\ T_1.plane=T_2.plane=T_3.plane=T_4.plane}}(\rho_{T_1}ODP \times \rho_{T_2}ODP \times \rho_{T_3}ODP \times \rho_{T_1}ODP)\right)$$

-- Now, ensure that plane exists in such set.

$$AtLeastFour = \emptyset$$